CarbonPass

Verified embedded-emissions passports for the EU Carbon Border Adjustment Mechanism, so clean producers stop paying for dirty defaults.

Track: Carbon Markets & Emissions Transparency

Live demo	https://carbonpass.vercel.app (Ethereum Sepolia testnet)
Source code	https://github.com/johnbosco-j/carbonpass
CarbonPassport	`0x2eD59Df13de482318Fa1dc61aAE5015cf7bE6a22` (Sepolia, source verified on Sourcify)
VerifierRegistry	`0x52ca9d789B4A7819FF453b887BfcE558b340Ba54`
Stack	Solidity / OpenZeppelin (ERC-1155), Hardhat, Next.js, viem, Google Gemini, ML-DSA-65 (FIPS 204), Merkle commitments

1. Problem statement & solution overview
2. Architecture diagram & technical specification
3. Security & data-integrity assessment
4. Climate impact assessment & carbon-reduction projections
5. Implementation, adoption strategy & scalability roadmap
6. Screenshots

Prototype on a public testnet. Companies, installations and verifiers in the demo are fictional. Emission factors and default values marked illustrative are placeholders, not official figures.

Problem statement & solution overview

CarbonPass: verified embedded-emissions passports for EU CBAM Track: **Carbon Markets & Emissions Transparency**

The problem

On **1 January 2026** the EU's Carbon Border Adjustment Mechanism (CBAM) entered its definitive period. Every tonne of steel, aluminium, cement, fertiliser, hydrogen or electricity imported into the EU now carries a carbon price, based on the emissions released to make it.

The legal obligation sits with the EU importer, but the data comes from the producer: a steel mill in Türkiye, an aluminium smelter in India. To declare **actual** emissions, that data must be checked by an accredited verifier. Otherwise the importer must use the EU's **default values, with a markup of 10% in 2026**. The charged share of emissions rises every year, from 2.5% in 2026 to 100% in 2034.

Today the data moves as spreadsheets and PDFs over email. This causes four failures:

1. **Clean producers are priced like dirty ones.** A mill that has invested in a cleaner process gets no reward if its data isn't trusted. Default values cut the link between carbon price and actual performance, and with it the incentive to decarbonise. This is the climate failure.
2. **Double counting.** Nothing stops the same verified low-carbon tonnes being promised to several buyers, the same reporting period being certified twice, or one report being reused across several declarations. This is the exact problem our track names.
3. **No verifiable chain of custody.** An importer receiving a PDF can't confirm it is the version the verifier approved, or that the verifier is accredited.
4. **Cost barrier for SMEs.** Turning invoices and meter logs into verifier-ready data is expensive consultant work. Small exporters, the members of chambers like the Istanbul Chamber of Commerce, are hit hardest.

This isn't hypothetical. For hot-rolled steel from Türkiye the EU default is **2.43 tCO_2e/t** (Implementing Reg. 2025/2621). Yet about **70% of Turkish steel is made in electric arc furnaces**, and the Aegean Ferrous and Non-Ferrous Metals Exporters' Association (İzmir) says the defaults don't reflect that lower-carbon production. They are asking for verified, plant-level data, and they note that the bodies to accredit CBAM verifiers aren't yet clearly defined. Our demo mill verifies at **0.337 tCO_2e/t**, using its pig-iron supplier's verified passport.

The same data gap blocks **Türkiye's ETS pilot** (monitoring, reporting and verification only in 2026) and **India's Carbon Credit Trading Scheme**. One trusted pipeline can serve all three.

Who it's for

USER	PAIN TODAY	WHAT CARBONPASS GIVES THEM
Producer (mill compliance officer, TR/IN)	Manual data prep; can't prove it's clean	AI-prepared, verifier-ready evidence; a passport they can share with buyers
Accredited verifier	Reviewing messy files; liability for sign-offs	Pre-checked evidence with anomaly flags; a signed, non-repudiable attestation
EU importer	Can't trust received data; default-value penalties	One QR scan to verify, plus the € saving; tonnes surrendered at declaration
Policymaker / registry	Fragmented MRV data	A consistent, auditable record that feeds national ETS and CBAM registries

The solution

1. **Capture (AI).** The producer uploads fuel and electricity invoices and the production log. An LLM extracts each quantity, with a citation to its document and page. A human confirms every value.
2. **Calculate.** A fixed calculator applies the CBAM method: direct emissions (fuels × emission factors, plus process inputs), indirect emissions (electricity × grid factor), divided by tonnes produced.
3. **Check (AI).** Robust statistical anomaly detection compares the month against the plant's own history and physical bounds, e.g. "intensity 38% below the 12-month median → verifier must review".
4. **Commit (blockchain).** The evidence is hashed into a salted Merkle tree. Only the root and the report hash go on-chain, so process data stays confidential.
5. **Attest (blockchain + cryptography).** An accredited verifier from the on-chain registry signs. This is a hybrid signature: the ECDSA transaction plus a **post-quantum ML-DSA** signature over the report, so it stays valid over multi-year record-keeping obligations. The contract then mints the batch: N tonnes at X $kgCO_2e/t$, as ERC-1155 tokens.
6. **Supply chain (blockchain).** Precursors carry their own passports. The mill's pig iron comes from a supplier's verified batch, and at issuance the contract **consumes (burns) those precursor tonnes** into the steel batch. It refuses a steel intensity lower than its own inputs' emissions. Embedded emissions propagate across companies on-chain, and precursor tonnes can't be claimed twice.
7. **Ship and surrender (blockchain).** Tonnes move to importers with shipments. When an importer files its CBAM declaration, it **burns** those tonnes. The contract makes double counting impossible:
    - you can't ship more tonnes than were verified
    - you can't certify an overlapping period twice
    - you can't reuse a report
    - you can't surrender tonnes you don't hold
8. **Prove.** A public passport page (from a QR code on the shipping documents) shows the verified intensity, the verifier, the full on-chain history, a **tamper check** for any evidence file, and the **cost saved versus default values**.

Why blockchain is necessary here

The parties sit in different jurisdictions, and none will run or trust the others' database. The core risk, the same verified tonnes claimed twice, is a double-spend problem. Public ledgers solve that by design. Verifier sign-offs need public, timestamped non-repudiation. Commitments (hashes, Merkle roots) give transparency **without** disclosing confidential plant data.

Why AI is necessary here

The bottleneck for SMEs is turning unstructured documents into structured, cited activity data. The bottleneck for verifiers is finding the few suspicious numbers in thousands. AI handles both, and **never** writes to the chain itself.

Track alignment

The Carbon Markets & Emissions Transparency track asks for blockchain-based verification, real-time emissions tracking, and smart contracts to prevent double counting. CarbonPass does all three for the world's first operating carbon border price. Its passports are also digital product passports for the *Sustainable Supply Chains* track.

Architecture & technical specification

1. System context

On-chain: registry of verifiers and installations, batch metadata, Merkle roots, report hashes, signature digests, tonne balances, the shipment and surrender history. **Off-chain:** raw documents, extracted activity data, salts, the full ML-DSA signature (about 3.3 KB). The producer holds these and shares them with the verifier. Anything off-chain can be checked against the on-chain commitments.

2. Main sequence

```
sequenceDiagram
    autonumber
    actor P as Producer
    participant W as Web app
    participant AI as Gemini API
    participant C as CarbonPassport
    actor V as Verifier
    actor A as Importer A
    actor B as Importer B

    P->>W: upload invoices + production log
    W->>AI: extract quantities (structured output, cited)
    AI-->>W: activity lines + source refs + confidence
    P->>W: confirm each value
    W->>W: calculate SEE, run anomaly checks, build Merkle tree
    W->>C: submitBatch(installation, CN, period, tonnes, intensities, root, reportHash)
    V->>W: review evidence + anomaly flags
    V->>W: ML-DSA sign(domain, batchId, root, reportHash)
    V->>C: attestAndIssue(batchId, keccak(pqSig))
    C-->>P: mint N tonnes (ERC-1155 id = batchId)
    P->>C: ship(A, batchId, 6,000 t, shipmentRef)
    P->>C: ship(B, batchId, 4,000 t, shipmentRef)
    P->>C: ship(B, batchId, 2,000 t)  ❌ reverts: insufficient balance
    A->>C: surrender(batchId, 6,000 t, declarationRef)  🔥 burn
```

Precursor chain (embedded emissions across companies)

CBAM counts the embedded emissions of relevant precursors (e.g. pig iron in steel). A supplier with its own verified passport ships those tonnes to the mill with the normal `ship()`. The mill declares them as inputs, and they're burned when its batch is verified:

```
sequenceDiagram
    autonumber
    actor S as Pig-iron supplier
    participant C as CarbonPassport
    actor P as Steel mill
    actor V as Verifier

    S->>C: submitBatch(pig iron, 5,000 t, 1,700 kg/t)
    V->>C: attestAndIssue(#4)  → 5,000 t to the supplier
    S->>C: ship(mill, #4, 1,300 t)
    P->>C: submitBatch(HRC, 10,000 t, 337 kg/t, precursors=[{#4, 1,300 t}])  (checked, nothing burned)
    V->>C: attestAndIssue(#5)  🔥 burns 1,300 t of #4, then mints 10,000 t of #5
    P->>C: submitBatch(August, precursors=[{#4, 1,300 t}])  ❌ InsufficientPrecursor(4, 0, 1300)
```

3. Smart contracts

Solidity 0.8.28 and OpenZeppelin Contracts 5.x. There's no ETH handling, so reentrancy has nothing to exploit.

`VerifierRegistry`

FUNCTION	WHO	EFFECT
`accredit(verifier, name, accreditationRef, pqPublicKeyHash)`	`ACCREDITOR_ROLE` (accreditation body)	Adds or updates an active verifier
`revoke(verifier, reasonHash)`	`ACCREDITOR_ROLE`	Deactivates a verifier
`isActive(verifier) → bool`	anyone	
`getVerifier(verifier) → Verifier`	anyone	

`CarbonPassport` (ERC-1155, token id = batch id)

```
struct PrecursorInput {
  uint256 batchId;           // an Issued upstream batch (e.g. pig iron)
  uint64  tonnes;            // tonnes of it embedded in this batch
}

struct Batch {
  address producer;          // installation operator
  bytes32 installationId;
  uint32  cnCode;            // e.g. 72083900 (hot-rolled flat steel)
  uint64  periodStart;       // unix seconds
  uint64  periodEnd;
  uint64  tonnes;            // product tonnes (integer)
  uint32  directKgPerT;      // kgCO2e per tonne
  uint32  indirectKgPerT;
  bytes32 dataRoot;          // Merkle root of salted evidence leaves
  bytes32 reportHash;        // sha256(canonical report JSON)
  address verifier;
  bytes32 pqSignatureHash;   // keccak256(ML-DSA-65 signature)
  uint64  submittedAt;
  uint64  issuedAt;
  uint64  surrendered;       // tonnes burned for declarations
  uint64  consumed;          // tonnes burned as a precursor of downstream batches
  Status  status;            // None, Pending, Issued, Rejected, Revoked
}
```

`BatchInput` (the `submitBatch` argument) carries the same fields plus `PrecursorInput[] precursors` (at most `MAX_PRECURSORS` = 8).

FUNCTION	WHO	CHECKS	EFFECT
`registerInstallation(id, operator, name)`	`REGISTRAR_ROLE`		Binds an installation to its operator
`submitBatch(...) → batchId`	the installation's operator	reportHash unused; period valid and after the last certified period; precursors: no duplicates, each Issued and fully held by the producer, intensity covers them	`Pending`; records the precursor claim (`PrecursorDeclared`), burns nothing
`attestAndIssue(batchId, pqSigHash)`	an active verifier	pending; verifier ≠ producer; period still free; precursor checks again	`Issued`; burns each precursor's tonnes from the producer (`PrecursorConsumed`), then mints `tonnes` to the producer
`rejectBatch(batchId, reasonHash)`	an active verifier	pending	`Rejected`
`ship(to, batchId, tonnes, shipmentRef)`	holder	balance ≥ tonnes; batch not revoked	Transfer + `Shipped` event
`surrender(batchId, tonnes, declarationRef)`	holder	balance ≥ tonnes	Burn + `Surrendered` event
`revokeBatch(batchId, reasonHash)`	issuing verifier or registrar	issued	`Revoked`; all transfers and burns frozen
`verifyDisclosure(batchId, key, value, unit, salt, proof) → bool`	anyone (view)		Merkle inclusion against `dataRoot`
`getPrecursors(batchId) → PrecursorInput[]`	anyone (view)		The inputs a batch declared; burned once it's Issued

Precursor errors: `InsufficientPrecursor(precursorId, held, needed)`, `PrecursorNotIssued(precursorId, status)`, `UnderstatedPrecursors(declaredKg, precursorKg)`, `DuplicatePrecursor(precursorId)`, `TooManyPrecursors(count, max)`.

Every precursor check runs at submit, so the producer finds out immediately. The checks run again at issuance, which is the authoritative point: a precursor can be revoked, or its tonnes moved, while the claim is pending. Submitting the claim is the producer's consent to the burn. Only the installation's operator can submit, and tonnes are only ever burned from that operator's balance. Precursors are burned *before* the downstream mint, so a contract producer's `onERC1155Received` hook can't move them away mid-issuance.

Invariants (covered by tests):

1. `totalSupply(id) + surrendered(id) + consumed(id) == tonnes(id)` for every issued batch, so tonnes are never created after issuance. A seeded random sequence of ships, claims, issuances, rejections and surrenders checks this, and that holder balances sum to the supply, after every step.
2. A `reportHash` can be used at most once.
3. For each installation and CN code, certified periods never overlap.
4. A producer can never verify its own batch. Only active accredited verifiers can issue.

5. A revoked batch can't move, and can't be consumed as a precursor.
6. **Precursor tonnes count once across companies.** They're burned exactly when the consuming batch is issued. A pending or rejected batch burns nothing. Consumed tonnes can never back a second batch.
7. **Embedded emissions can't be understated:** `directKgPerT × tonnes ≥ Σ precursor.tonnes × precursor.directKgPerT` (integer form, no rounding). Precursor intensities are immutable once issued.
8. **The supply-chain graph is acyclic:** a precursor must already be Issued when the claim is submitted, so its id is always lower than the batch consuming it.

4. Evidence commitment

Leaves: `[key, value, unit, salt]`, encoded as `(string, string, string, bytes32)` in an OpenZeppelin `StandardMerkleTree`. Each leaf is `keccak256(bytes.concat(keccak256(abi.encode(...))))`. Examples:

KEY	VALUE	UNIT
`fuel.natural_gas`	`118400`	`GJ`
`electricity.grid`	`46250`	`MWh`
`input.electrodes`	`172`	`t`
`production.hrc`	`10000`	`t`
`doc.gas_invoice_2026-07.pdf`	`<sha256 hex>`	`sha256`

- **Salt:** a random 32 bytes per leaf. Activity values have low entropy, and without salts someone could brute-force a leaf.
- **Tamper check:** a file dropped on the passport page is re-hashed (SHA-256). It matches only if a `doc.*` leaf with that hash proves into `dataRoot`.
- **Report hash:** SHA-256 of the canonical JSON report (sorted keys, no whitespace).

5. Hybrid verifier signature

- **ECDSA (secp256k1):** the verifier's transaction calling `attestAndIssue`. It's native and gives immediate non-repudiation.
- **ML-DSA-65 (FIPS 204)** via `@noble/post-quantum`. The message is the domain-separated tuple `("CarbonPass/v1", chainId, contract, batchId, dataRoot, reportHash)`. The signature is stored off-chain, and `keccak256(sig)` goes on-chain. The verifier's PQ public-key hash is registered in `VerifierRegistry`.
- **Why:** CBAM and ETS records must stay attributable for years. Hybrid signing keeps attestations unforgeable even if secp256k1 is broken during that period. It's crypto-agile: the scheme is versioned in the domain tag.

6. CBAM calculation (simplified, in `web/lib/cbam.ts`)

```
direct_t   = Σ fuel_GJ × EF_fuel (tCO2/GJ)  +  Σ process_input_t × EF_input (tCO2/t)
indirect_t = electricity_MWh × grid_EF (tCO2/MWh)
SEE_direct   = direct_t / production_t          (tCO2e per t product)
SEE_indirect = indirect_t / production_t
chargeable   = SEE_direct (+ SEE_indirect for cement, fertilisers)
cbam_cost(y) = tonnes × chargeable × CBAM_factor(y) × EU_ETS_price
default_cost(y) = tonnes × default_value × (1 + markup(y)) × CBAM_factor(y) × EU_ETS_price
```

All factors live in `web/data/factors.json`, each with a source and a verify note. Purchased pig iron is included as a **precursor** (`precursor_t × SEE_precursor`, added to direct). When the mill holds a verified pig-iron passport, `SEE_precursor` is that passport's on-chain intensity, and Submit declares the tonnes so the contract burns them at issuance. Otherwise it falls back to the EU default value (1.85 tCO_2e/t for Türkiye). In the demo, 1,300 t at a verified 1.700 instead of 1.85 takes July from 0.356 to 0.337 tCO_2e/t. Only direct precursor emissions are enforced on-chain, because direct emissions are the chargeable scope for iron and steel. The evidence report commits to the precursor claim, so the verifier's signature covers it.

7. Anomaly checks (in `web/lib/anomaly.ts`)

- **Robust z-score** against the installation's 12-month history: `z = 0.6745 × (x − median) / MAD`. Flag if |z| > 3.5. This is Iglewicz & Hoaglin's modified z-score.
- **Physical bounds** per route, e.g. EAF electricity 0.35–0.75 MWh/t. Out of range means a flag.
- **Cross-document consistency:** e.g. the invoiced MWh vs. the meter log, flagged above 2% difference.
- Flags are advisory. The verifier must acknowledge each one before signing.

8. AI extraction (in `web/app/api/extract`)

- The Gemini API (Interactions API, `gemini-3.8-flash`, free tier) gets each PDF and a **JSON schema** (generated from a Zod schema) for the output. Each line has: `{ key, value, unit, period, source: { file, page, quote }, confidence }`.
- Documents are treated as untrusted data. The model has no tools, and its output is validated against the schema and unit whitelist.
- Without an API key, pre-recorded sample outputs in `web/data/extractions/` are returned (hand-prepared, labelled as such in the UI), so the demo always works.

9. Web app

ROUTE	ROLE	PURPOSE
`/`	public	Problem, live chain stats, start demo
`/producer`	producer	Evidence upload → extraction → calculation → checks → submit
`/verifier`	verifier	Queue → evidence + flags → sign → issue (stamp)
`/shipments`	producer	Tonnage gauge, ship to A / B, over-ship revert
`/importer`	importer	Holdings, surrender at declaration
`/passport/[id]`	public	QR, intensity, verifier, precursors + supply-chain graph, timeline, tamper check, cost model

API routes (`/api/tx/*`) sign with **demo role wallets** held as server env vars on testnet. In production each party signs with its own wallet or HSM. The UI shows the acting address on every action.

10. Public-RPC consistency (read-your-writes)

Free Sepolia RPCs are load-balanced, so a read right after a write can hit a node a block behind. Two problems follow: "batch doesn't exist" errors, and false "insufficient balance" predictions. The app handles this:

- Simulation and gas estimation are **pinned to the newest block known to contain our own writes**. The browser passes its last-seen block as `minBlock`.
- Lag-type reverts (`UnknownBatch`, `NotPending`) are retried briefly.

- For forced on-chain sends, the **receipt is the source of truth**, never the prediction.
- Nonces are tracked per role, so a stale node can't hand back an old nonce.

11. Deployment

- **Contracts:** Hardhat, `pnpm --filter contracts deploy:sepolia`, then source verification on Etherscan.
- **Seed:** register the verifier and both installations (the steel mill and its pig-iron supplier), plus 3 historical issued steel batches (so the passport timeline and history aren't empty), and the supplier's June 2026 pig-iron batch (#4, 5,000 t at 1,700 $kgCO_2e/t$) with 1,300 t shipped to the mill for the July demo.
- **Web:** Vercel (https://carbonpass.vercel.app). Env vars: `NETWORK`, `RPC_URL`, demo role keys, `VERIFIER_PQ_SEED`, optional `GEMINI_API_KEY` / `GEMINI_MODEL`. Contract addresses come from `web/lib/chain/deployments.json`.

Security & data-integrity assessment

What a blockchain can and can't guarantee here:

- **It can prove** that a record hasn't changed since it was committed, who signed it and when, and that tonnes were never counted twice.
- **It can't prove** on its own that the underlying meter reading was true. That's the "oracle problem".

CarbonPass handles that second gap with layered controls: AI cross-checks, statistical anomaly detection, an accredited human verifier, and revocation. We say this plainly rather than claiming "blockchain makes data true".

1. Assets

ASSET	CONFIDENTIALITY	INTEGRITY	AVAILABILITY
Plant activity data (fuel, electricity, production)	**High** (commercially sensitive)	High	Medium
Verified intensity per batch	Public	**High**	High
Tonne balances (ownership of verified tonnes)	Public	**Critical**	High
Verifier attestation	Public	**Critical**	High
Role keys	**Critical**	Critical	Medium

2. Threat model (STRIDE)

#	THREAT	CATEGORY	MITIGATION IN CARBONPASS	RESIDUAL RISK
T1	Producer falsifies activity data (garbage in)	Tampering	Every value is cited to a source document. Anomaly detection runs against history and physical bounds, with cross-document checks. An accredited verifier must acknowledge every flag	Collusion with the verifier (T6)
T2	The same period is certified twice	Tampering	The contract enforces non-overlapping periods per installation and CN code	None at contract level
T3	The same report is reused for another batch	Tampering	`reportHash` uniqueness	None at contract level
T4	Verified tonnes are sold to several importers (double counting)	Tampering / Repudiation	ERC-1155 balance conservation. Over-shipping reverts	None at contract level
T5	Tonnes are reused across several CBAM declarations	Tampering	`surrender` burns them. The declaration reference is logged	The declaration itself sits in the EU registry; an integration hook is on the roadmap
T6	A rogue or bribed verifier	Spoofing / Elevation	Only `ACCREDITOR_ROLE` adds verifiers. Revocation. Batches can be revoked and then frozen. Every attestation is publicly attributable	Detection depends on audits; on-chain data makes audits cheap
T7	A producer verifies its own batch	Elevation	`verifier ≠ producer` check	None at contract level

#	THREAT	CATEGORY	MITIGATION IN CARBONPASS	RESIDUAL RISK
T8	An impersonated installation (someone else submits)	Spoofing	`registerInstallation` binds an installation to its operator. Only the operator can submit	Registrar key compromise (T10)
T9	Evidence document swapped after issuance	Tampering	Document SHA-256 values are Merkle leaves. The passport tamper check fails on any changed byte	None
T10	Key compromise (producer, verifier, admin)	Spoofing	Role separation (AccessControl). Revocation paths. Production: multisig for admin roles, HSM for verifiers	Window between compromise and revocation
T11	Confidential data leaks from on-chain commitments	Information disclosure	Only roots and hashes on-chain. **Salted leaves** stop dictionary attacks on low-entropy values. Selective disclosure reveals one field at a time	Batch-level intensity is public by design (the importer needs it)
T12	Future quantum attacks forge old attestations	Spoofing / Repudiation	**Hybrid signature:** ECDSA plus ML-DSA-65 (FIPS 204). The contract stores the PQ signature's hash and emits the full signature; the verifier's PQ public key is registered on-chain. **The contract doesn't verify ML-DSA itself** (too costly on today's EVM). Anyone verifies it off-chain from chain data alone, and the passport page flags a batch whose PQ signature fails	If secp256k1 were broken, a forged transaction could still be *accepted* by the contract, but its attestation would visibly fail the PQ check. Enforcing it on-chain needs a PQ precompile or an L1 migration (roadmap)
T13	AI extraction (Gemini) hallucinates or misreads a value	Tampering (accidental)	Schema-validated output, a unit whitelist, a citation (file + page + quote) per value, a confidence score, and **mandatory human confirmation**. The AI never writes on-chain	Human confirmation fatigue; low-confidence values are highlighted
T14	Prompt injection inside an uploaded document	Elevation	Documents are treated as data. The model has no tools, no secrets in the prompt, and output validated against the schema	None meaningful
T15	Smart-contract bugs	Tampering	OpenZeppelin bases, custom errors, no ETH handling, invariant-focused tests. Static analysis (Slither) is on the roadmap before mainnet	Unaudited prototype
T16	Demo API abuse drains testnet gas	Denial of service	Demo-only role wallets, rate limiting, testnet only	Testnet only
T17	Supply-chain double counting: the same verified precursor tonnes (e.g. pig iron) are claimed by two downstream products, or a mill understates what its precursors embed	Tampering	Precursor tonnes are ERC-1155 balances of the supplier's batch, burned from the mill's balance when its batch is issued, so they can back only one product (`InsufficientPrecursor`). Checked at submit and again at issuance; a rejected batch burns nothing. The downstream intensity must cover its precursors (`UnderstatedPrecursors`). Only issued, unrevoked precursors count (`PrecursorNotIssued`). Inputs are capped (`MAX_PRECURSORS`) to bound gas	A precursor revoked *after* consumption doesn't cascade automatically: the downstream passport shows the revoked input, and the downstream verifier or registrar can revoke it. Tonnes are integers, so a fractional tonne of precursor rounds

3. Contract invariants (tested)

1. Tonnes are conserved: supply plus surrendered plus consumed-as-precursor equals issued (also checked after every step of a seeded random sequence of operations).
2. Report hashes are unique.
3. Certified periods don't overlap.
4. Only active accredited verifiers can issue. A producer can't self-verify.
5. Revoked batches are frozen.
6. Disclosure proofs verify only for leaves in the committed root.
7. Precursor tonnes are burned once, at the consuming batch's issuance. Reuse, over-claiming, revoked or pending precursors, and an understated downstream intensity all revert. A rejected batch consumes nothing.

Test file: `contracts/test/CarbonPassport.test.ts`.

4. Cryptography choices

USE	PRIMITIVE	WHY
Document fingerprints, report hash	SHA-256	Standard for document integrity and auditor-familiar
Evidence commitment	keccak256 Merkle tree (OpenZeppelin `StandardMerkleTree`) with 32-byte salts	Cheap on-chain verification; double-hashed leaves resist second-preimage attacks
Transaction / attestation signature	ECDSA secp256k1	Native to EVM
Long-lived attestation signature	**ML-DSA-65** (NIST FIPS 204)	Post-quantum security; crypto-agile domain tag `CarbonPass/v1`

5. Privacy & data protection

- There's no personal data on-chain. Installations are pseudonymous IDs, and entity names are held by the registry operator.
- Data minimisation: the chain holds only what an importer needs (intensity, tonnes, verifier) plus commitments.
- Producers control disclosure. Verifiers get the full bundle; importers get the passport and optional per-field proofs.

6. Responsible AI

- **Human in the loop:** AI suggests, humans confirm, verifiers sign.
- **Explainability:** every extracted value shows its source quote. Every anomaly flag shows its statistic and threshold.
- **Fairness:** anomaly thresholds compare an installation to *its own* history, not to a peer group that might penalise atypical-but-legitimate plants.
- **Accountability:** every on-chain action is attributable to a role address.

7. Known limitations of the prototype

Testnet only. Demo custodial keys. Illustrative emission factors. No external audit. The verifier workflow is simplified (no site visit). These are listed openly in the README.

- **Not an official declaration.** The demo verifier is fictional and holds no accreditation. A testnet passport is not a CBAM declaration and is not recognised by any EU registry. The prototype demonstrates the intended workflow.
- **Scope of double-counting protection.** The contract makes double counting impossible *within CarbonPass*. Preventing the same tonnes being claimed in a different system (another registry, a spreadsheet declaration) needs integration with the EU CBAM Registry and national registries (roadmap phase 2).
- **Data truth.** The chain proves who signed what, when, and that tonnes are conserved. It doesn't prove a meter reading is true; the accredited verifier and the anomaly checks address that.

Climate impact assessment

> **Interactive version: /impact.** The page reads live totals from the chain (verified tonnes, tonnage-weighted intensity vs the EU default, overstatement corrected, tonnes surrendered, € saved at the 2026 CBAM factor). It also has a scenario simulator for 2026–2034 (mills, exports, intensity, abatement, ETS price, Türkiye or India), the key-insight chart (a 0.1 tCO_2e/t cut is worth €0 on default values but more each year on verified data), policy context with sources, and the limitations below. Code: `web/app/impact`, `web/components/impact`, `web/lib/impact.ts` (the hand-checked scenario is in a comment there).

1. Theory of change

CarbonPass doesn't capture carbon itself. Its impact comes from **re-connecting the carbon price to actual performance**:

```
Unverifiable data → default values → bill doesn't depend on real emissions → marginal value of
abatement ≈ €0
Verified data     → actual values  → every tCO2e/t cut lowers the bill  → marginal value of
abatement > €0
```

Under default values, a mill that cuts its emissions pays the same CBAM bill as one that doesn't. With trusted, verified data, every tonne of CO_2 abated is money saved at the border. That incentive grows each year as the CBAM factor phases in, from 2.5% in 2026 to 100% in 2034.

Three levels of benefit, and how solid each one is:

LEVEL	WHAT IT MEANS	STATUS IN THIS PROTOTYPE
1. Compliance	The producer can declare *actual* verified emissions instead of the default value	**Demonstrated:** the full flow runs on Sepolia
2. Financial	The importer's CBAM bill is computed on 0.337 instead of 2.43 × mark-up	**Calculated** from the published rules (phase-in factor, default values, mark-ups); the ETS price is an assumption
3. Climate	The restored price signal leads plants to cut emissions	**A hypothesis, to be measured in a pilot.** Not demonstrated by the prototype

Level 3 is the one that matters for the climate, so we keep it honest. CarbonPass doesn't claim measured reductions; it restores the price signal that carbon markets rely on. There's strong evidence that signal works: the EU ETS cut about **1.2 billion tonnes of CO_2 (3.8%) between 2008 and 2016**, even at low prices (Bayer & Aklin, *PNAS* 117(16), 2020). Under default values, CBAM sends exporters no such signal. Verified data turns it on.

Secondary effects:

- **Market integrity:** double counting of low-carbon claims becomes technically impossible. That protects the credibility of "green steel" premiums.
- **Cheaper MRV at scale:** the same pipeline produces data for **Türkiye's ETS pilot** (MRV-only in 2026) and **India's CCTS**. Better measurement is the precondition for meaningful caps.
- **Inclusion:** SMEs that can't afford consultants can still reach actual-value reporting, so climate policy doesn't simply push them out of the EU market.

2. Quantification method

Per installation, per year `y`:

```
savings(y)            = T × [DV × (1 + m(y)) − SEE] × F(y) × P
abatement_value(y)    = ΔSEE × T × F(y) × P          (the € value of cutting intensity by ΔSEE)
emissions_reduced(y)  = ΔSEE × T_total               (tCO2e/yr, if the incentive triggers
abatement ΔSEE)
```

`T` = tonnes exported to the EU · `DV` = default value · `m` = default markup · `SEE` = verified specific embedded emissions · `F` = CBAM factor · `P` = EU ETS price · `T_total` = total production.

3. Illustrative scenario

> The installation is fictional and its intensity comes from the demo's sample evidence. The **default value is the EU's published CBAM default for hot-rolled flat steel from Türkiye** (2.43 tCO_2e/t, direct emissions, Implementing Reg. (EU) 2025/2621 Annex I). Check it against the July 2026 corrections (Reg. 2026/1740) before quoting.

Assumptions: an EAF mill in İzmir exports **100,000 t/yr** of hot-rolled coil to the EU. Its verified SEE (direct + precursors) is **0.337 tCO_2e/t**, the figure the demo calculates with its pig-iron supplier's verified passport linked. Default value **2.43 tCO_2e/t**, with a mark-up of **10% (2026), 20% (2027), 30% from 2028**. EU ETS price **€80/t** (an assumption; adjustable in the app).

YEAR	CBAM FACTOR	BILL ON DEFAULT VALUES	BILL ON VERIFIED DATA	SAVING
2026	2.5%	€534,600	€67,400	**€467,200**
2028	10%	€2,527,200	€269,600	**€2,257,600**
2030	48.5%	€12,256,920	€1,307,560	**€10,949,360**
2034	100%	€25,272,000	€2,696,000	**€22,576,000**

This matches what Turkish exporters themselves report. About 70% of Türkiye's steel is made via the electric-arc-furnace route, and the Aegean Ferrous and Non-Ferrous Metals Exporters' Association says the default values don't reflect that lower-carbon production (SteelOrbis, 1 Apr 2026). Verified, plant-level data is exactly what they are asking for.

The abatement incentive (what cutting 0.1 tCO_2e/t is worth on 100 kt of exports):

YEAR	UNDER DEFAULT VALUES	UNDER VERIFIED DATA
2026	€0	€20,000 / yr
2030	€0	€388,000 / yr
2034	€0	€800,000 / yr

Emissions scenario for a pilot. Take 5 installations, each producing 200 kt/yr. Suppose the restored incentive drives a 0.05 tCO_2e/t cut (e.g. switching reheating-furnace fuel, or optimising electrode and charge-carbon use). That's 5 × 200,000 × 0.05 = **50,000 tCO_2e/yr avoided**. This is a scenario showing the order of magnitude, not a forecast.

The /impact simulator opens on this pilot, treating all 200 kt per mill as EU exports. By default the cut phases in with the CBAM factor (the share of the carbon price actually charged): 1,250 tCO_2e in 2026, 50,000 tCO_2e/yr by 2034, and **204,500 tCO_2e cumulative over 2026–2034**. If the full cut happens from 2026, that becomes 450,000 tCO_2e. Over the same years the pilot's CBAM bills are €921 million lower on verified data than on defaults.

4. Indicators we'd track in a pilot

- Share of shipments declared on actual vs. default values
- Verified SEE trend per installation over time
- Double-counting attempts blocked (reverted transactions)
- Time from data upload to verifier sign-off
- Cost per verified passport vs. consultant baseline

5. Limitations & risks to impact

- **Resource shuffling:** a producer could send its cleanest output to the EU and the rest elsewhere. Installation-level batches make the mass balance visible but don't stop this. Policy has to.
- **Verification quality:** impact depends on honest verifiers. We reduce the cost of auditing them; we don't remove the need.
- **Scope:** for steel and aluminium only direct emissions are chargeable, so electricity-side improvements show up in CBAM only for cement and fertilisers. We still report indirect emissions for buyers' Scope 3.
- **A saving is not an emissions cut:** the € saving is a smaller border bill. The climate benefit appears only if that price signal leads plants to abate. "Overstatement corrected" (tonnes × (default – verified)) fixes the accounting; it removes no CO_2.
- **Additionality goes both ways:** counting avoided emissions on EU exports only understates a plant-wide process change. Attributing every cut to the price signal overstates it, because some plants would have abated anyway.
- The numbers above are sensitive to the default values and the ETS price. The app's cost model and the /impact simulator let users change both.

Implementation, adoption & scalability roadmap

Adoption strategy

Who pays and why:

- **Producers** pay per verified passport. That's far cheaper than the default-value penalty (see IMPACT.md) and than consultant-prepared reports.
- **Verifiers** get a free workspace. Pre-checked evidence means less review time per batch.
- **Importers** verify for free. Every passport they scan pulls their suppliers onto the network.

Go-to-market:

1. **Chambers & export associations.** The Istanbul Chamber of Commerce (a hackathon partner) and Indian export promotion councils reach thousands of CBAM-exposed SMEs.
2. **Verifiers as channel.** One accredited verifier brings all of its clients.
3. **EU importer pull.** Importers want actual values to cut their certificate bill, so they ask suppliers for a CarbonPass link.

Friction we remove: no crypto wallet is needed for importers or the public. Producers and verifiers can use managed keys. The outputs export to the existing CBAM communication template.

Phases

PHASE	WHEN	SCOPE	EXIT CRITERIA
0: Prototype	Hackathon (Oct 2026)	Steel (EAF), one installation, testnet, full flow	Working demo; contracts tested
1: Pilot	Q1–Q2 2027	3–5 Turkish / Indian installations, 1 verifier, 2 EU importers. A permissioned EVM (Hyperledger Besu) or public L2. CBAM template export. Slither + external audit	First 2026 CBAM declarations (due 30 Sep 2027) filed on verified data
2: Registry interop	H2 2027	Feeds to the EU CBAM Registry, the Türkiye ETS registry (EXIST) and India's CCTS. W3C Verifiable Credentials. EBSI-compatible issuer identities	One dataset → three compliance regimes
3: All CBAM goods	2028	Cement and fertilisers (indirect emissions chargeable) → **retire renewable-electricity certificates into batches** so they can't be double-claimed. Aluminium. Hydrogen	Multi-sector coverage
4: Supply-chain chaining	2028+	**Precursor passports** (pig iron → steel already works in the prototype): extend to alumina, clinker and ammonia, so any precursor batch is consumed into a downstream batch and embedded emissions propagate automatically. EU Digital Product Passport (ESPR) alignment	End-to-end product carbon footprint

Scalability

DIMENSION	APPROACH
Transactions	One batch per installation × product × period (not per shipment): thousands, not millions, per year. Shipments are cheap token transfers. An L2 or permissioned chain keeps costs below €0.01

DIMENSION	APPROACH
Data volume	Raw data stays off-chain. On-chain storage per batch is about 0.5 KB
Confidentiality at scale	Today: salted Merkle commitments + selective disclosure. Next: **zero-knowledge proofs** that the SEE was computed correctly from committed data, without revealing it
Interoperability	Standard token interface (ERC-1155), W3C VC export, CBAM template mapping, ISO 14064-3 aligned verification statements
Governance	Accreditation bodies hold `ACCREDITOR_ROLE`, national registries hold `REGISTRAR_ROLE`, multisig admin, upgrade via a versioned redeploy + migration
Crypto-agility	Versioned signature domain tags. ML-DSA today, swappable for other NIST PQC schemes

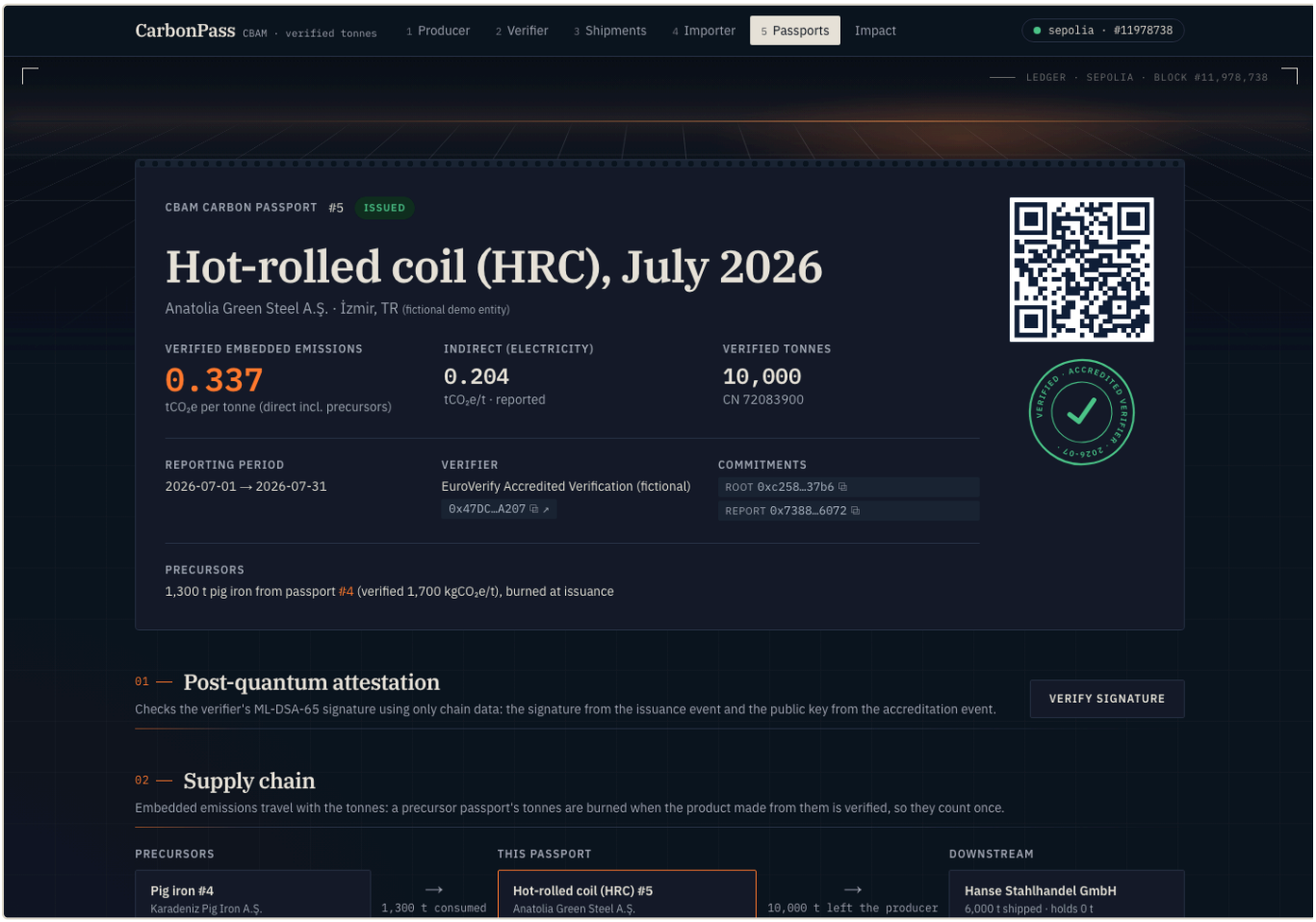
Standards alignment

- EU CBAM Regulation (EU) 2023/956 and its implementing regulations (methodology, default values, verification)
- GHG Protocol / ISO 14064-1 (quantification), ISO 14064-3 (verification), ISO 14067 (product carbon footprint)
- W3C Verifiable Credentials 2.0 (passport export), EU Digital Product Passport (ESPR)
- NIST FIPS 204 (ML-DSA)

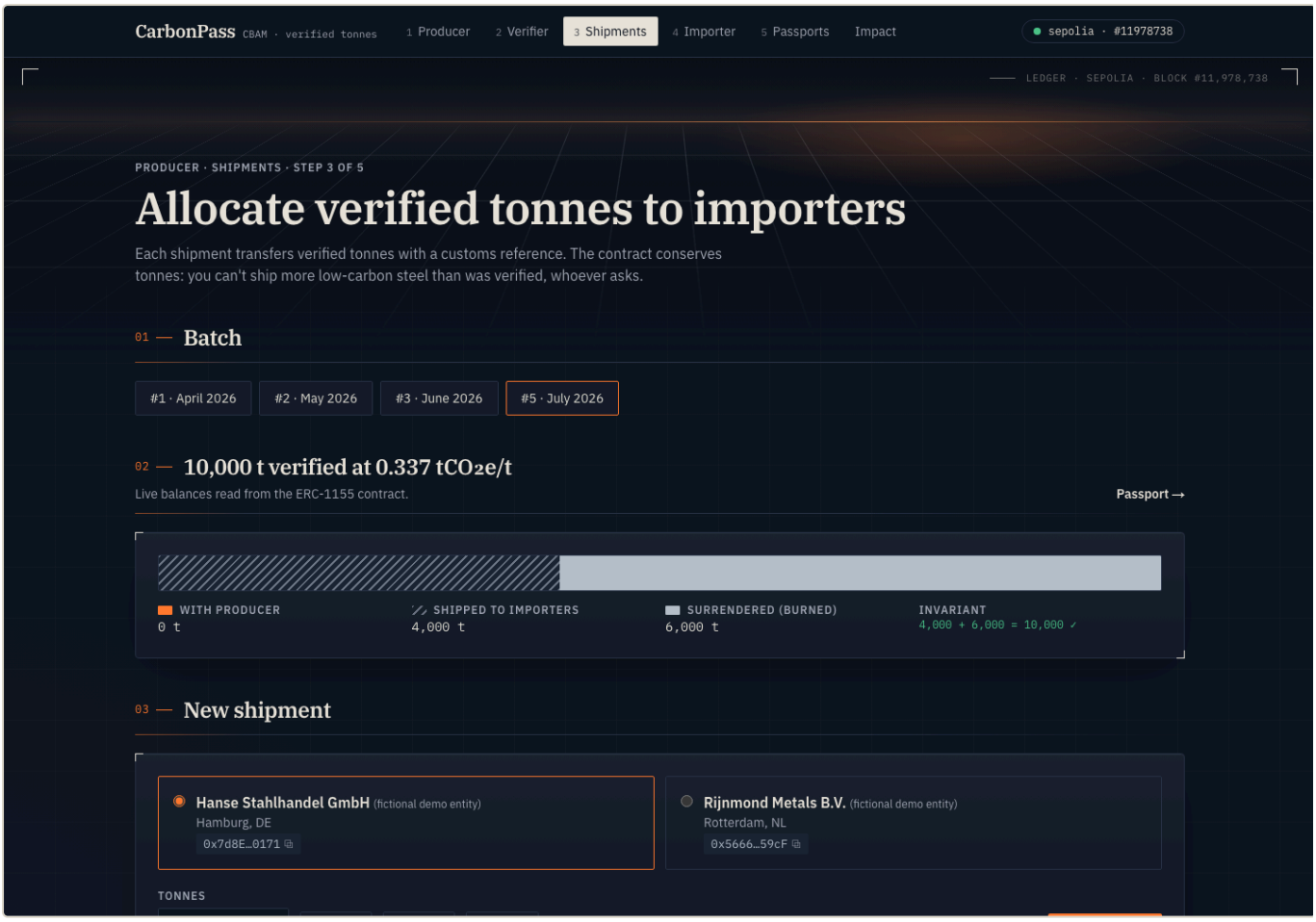
Sustainability of the solution itself

Proof-of-stake chains (Ethereum, its L2s) and permissioned networks use negligible energy per transaction. The system adds far less CO_2 than a single avoided default-value overestimate.

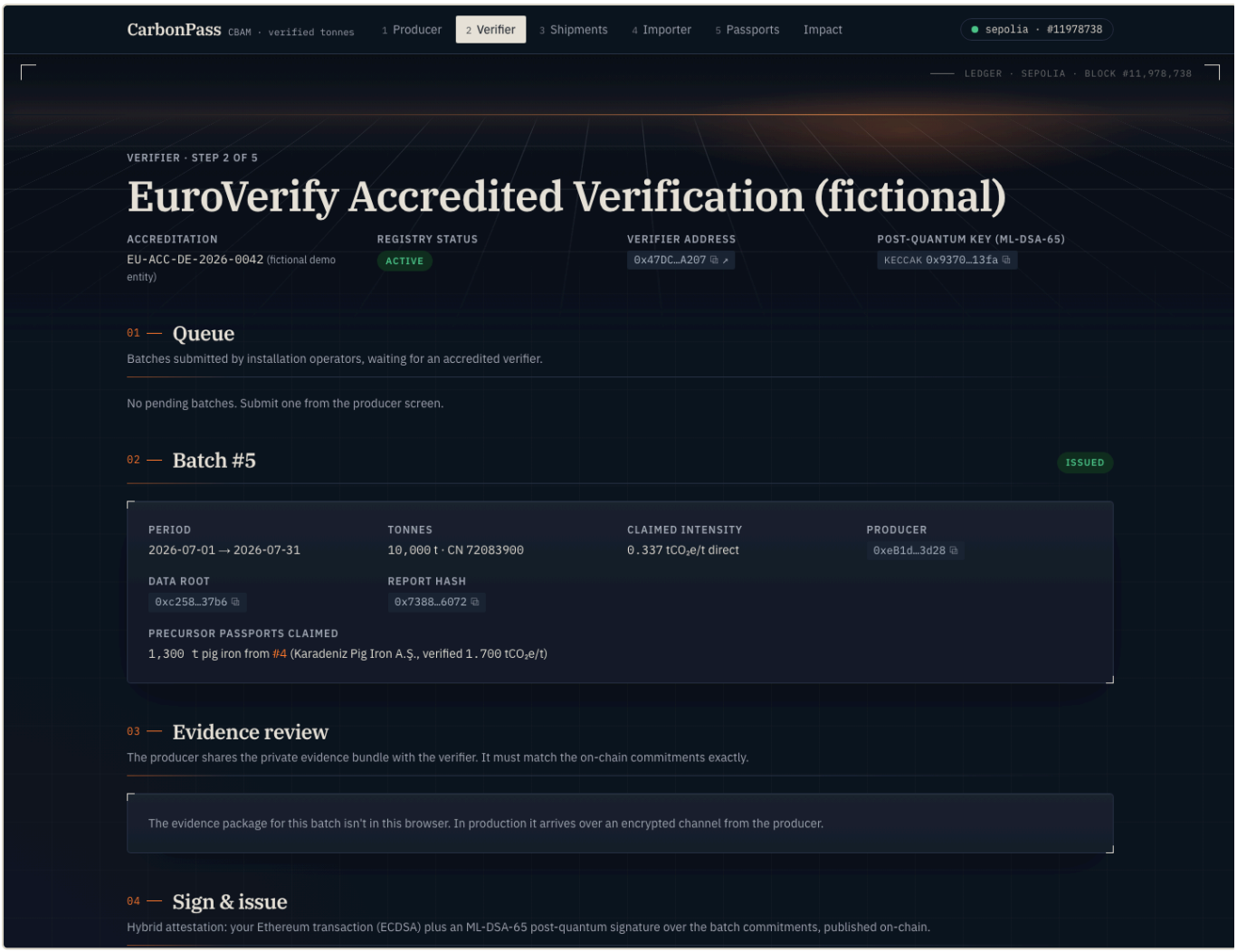
6. Screenshots



Public passport (Sepolia batch): verified intensity, QR, verifier stamp, commitments



Conservation gauge: every verified tonne is with the producer, an importer, or burned



Verifier: evidence matches on-chain commitments; hybrid ECDSA + ML-DSA-65 sign-off



Landing page with live chain statistics